You must use quotes if you are using interpolation to build the array index:

```
$age["Clone{$number}"]
```

Although sometimes optional, you should also quote the key if you are interpolating an array lookup to ensure that you get the value you expect:

```
// these are wrong
print "Hello, {$person['name']}";
print "Hello, {$person["name"]}";
```

5.2.4 Storing Data in Arrays

Storing a value in an array will create the array if it did not already exist, but trying to retrieve a value from an array that has not been defined would not create the array. *For example*:

```
// $addresses not defined before this point
echo $addresses[0];                  // prints nothing
echo $addresses;                     // prints nothing

$addresses[0] = "spam@cyberpromo.net";
echo $addresses;                        // prints "Array"
```

Using simple assignment to initialize an array in your program can lead to code like this:

```
$addresses[0] = "spam@cyberpromo.net";
$addresses[1] = "abuse@example.com";
$addresses[2] = "root@example.com";
```

That's an indexed array, with integer indices beginning at 0. Here's an associative array:

```
$price['gasket'] = 15.29;
$price['wheel'] = 75.25;
$price['tire'] = 50.00;
```

An easier way to initialize an array is to use the array() construct, which builds an array from its arguments. This builds an indexed array, and the index values (starting at 0) are created automatically:

```
$addresses = array("spam@cyberpromo.net", "abuse@example.com", "root@example.com");
```

To create an **associative array** with array(), use the => symbol to separate indices (keys) from values:

```
$price = array(
  'gasket' => 15.29,
  'wheel'  => 75.25,
  'tire'   => 50.00
);
```

Notice the use of whitespace and alignment. We could have bunched up the code, but it would not have been as easy to read (this is equivalent to the previous code sample), or as easy to add or remove values:

;(price = array('gasket' => 15.29, 'wheel' => 75.25, 'tire' => 50.00$